Processes, descriptors and pipes

Resources

CPU time, memory, devices

→

Shell

Commands become argv + streams

→

Files

Names, inodes, data, permissions

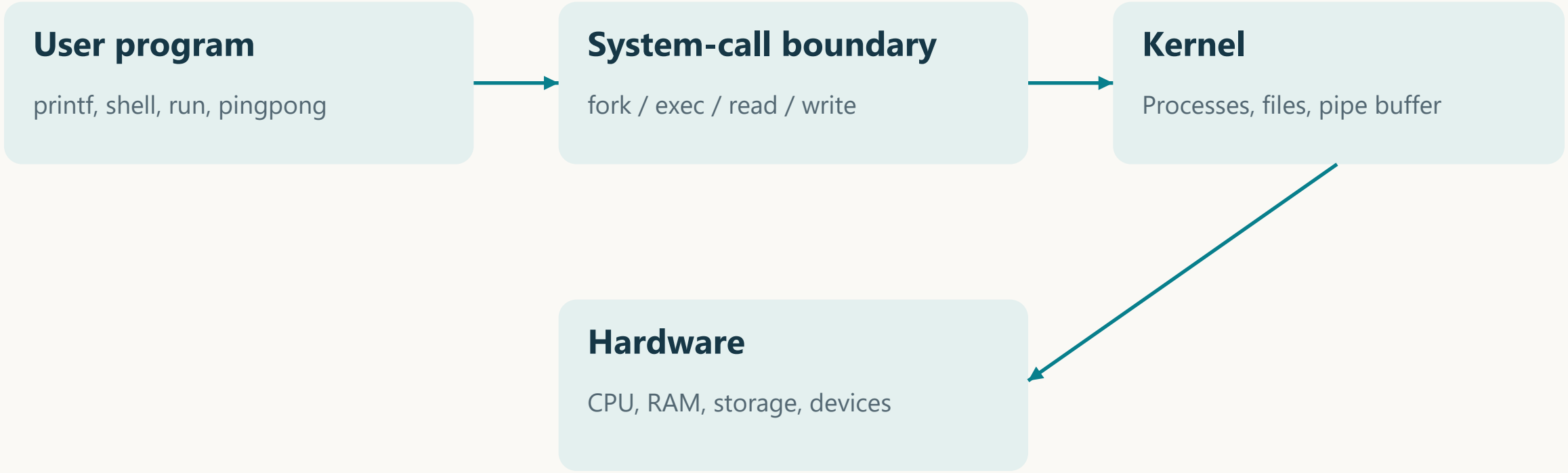
→

Processes

Create, replace, connect, reap

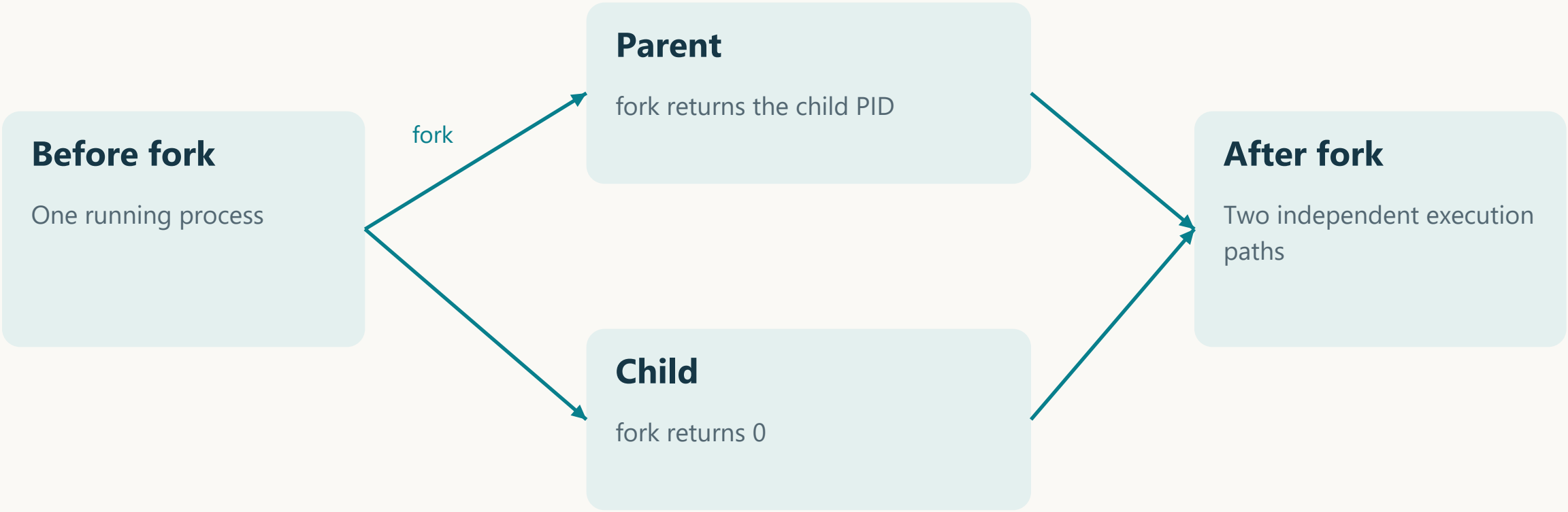
Labs 7 and 8 reveal the mechanism behind the commands used in the earlier labs.

A program asks; the kernel acts



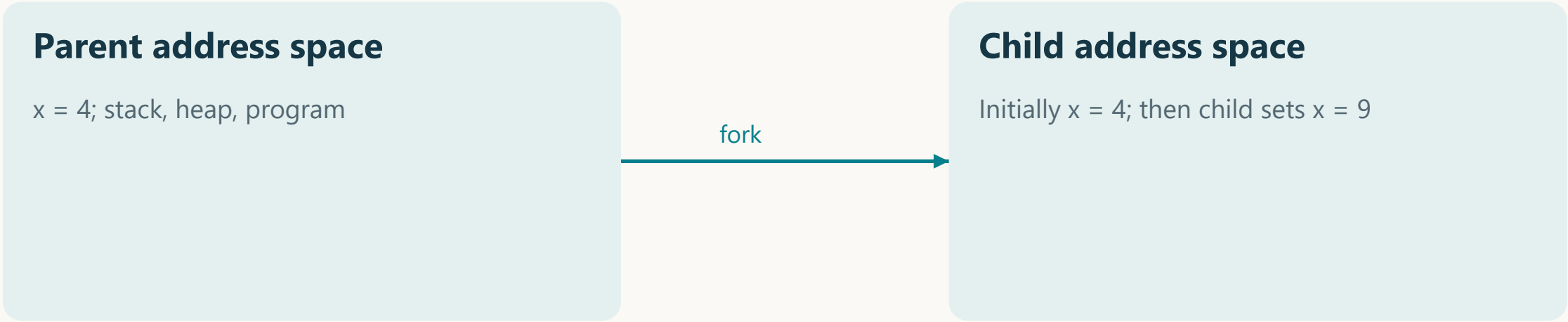
A system call is a controlled request for a privileged kernel service.

fork: one call, two return paths



The scheduler chooses which runnable process executes next.

Copied memory, shared open objects



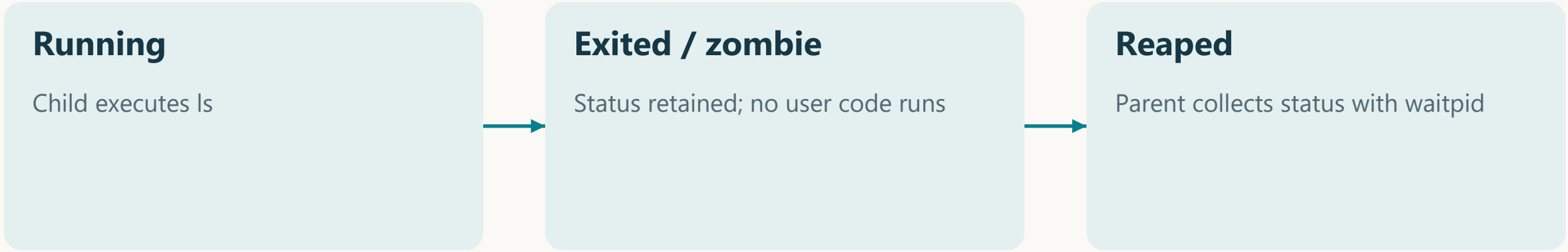
The child changing x does not change the parent's x. Descriptors are copied too, but may refer to the same underlying open file or pipe.

exec changes the program, not the PID



Success replaces the program and never returns. Failure returns to run, which reports the error and exits.

An exited child still needs collecting



sleep(20) delays the parent. waitpid removes the completed child record.

Redirection changes a descriptor

Child descriptor table

```
0  ->  terminal input
1  ->  terminal output
2  ->  terminal errors
```

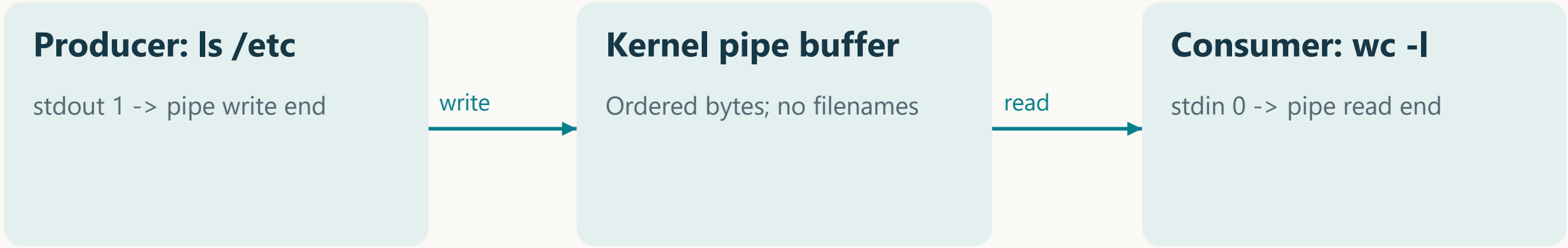
lowest free = 1

After close(1); open(...)

```
0  ->  terminal input
1  ->  out.txt
2  ->  terminal errors
```

ls still writes to descriptor 1. The descriptor now points to a file.

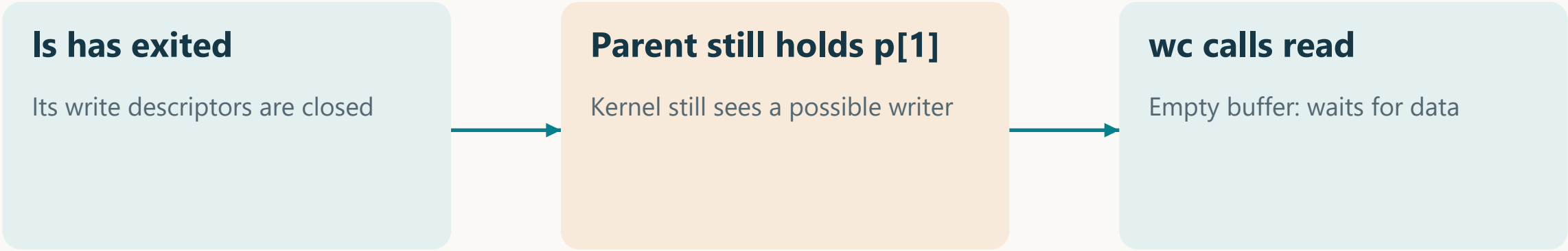
A pipeline is wiring plus processes



Parent closes both ends. Each child closes the unused original descriptors after dup.

Bytes flow from producer to consumer; the shell supplies the wiring.

Why the broken pipeline hangs



EOF requires BOTH an empty buffer and zero open references to the write end.

close the final writer -> read returns 0 -> wc prints its count -> wait completes

Lab 7: the observed behavior

Program replacement

ls: 0 | date: 0 | missing: 127

Zombie and redirection

Z+ before waitpid | open returned fd 1

Pipeline comparison

172 = 172 | optional three-stage: 108 = 108

Observed 19 October 2026 | Ubuntu 24.04 WSL2 | GCC 13.3.0

The same interface inside xv6

user/run.c

Calls fork(), exec(), wait()

→

System-call stub

a7 = call number; ecall

→

Trap + dispatch

usertrap -> syscall -> sys_*

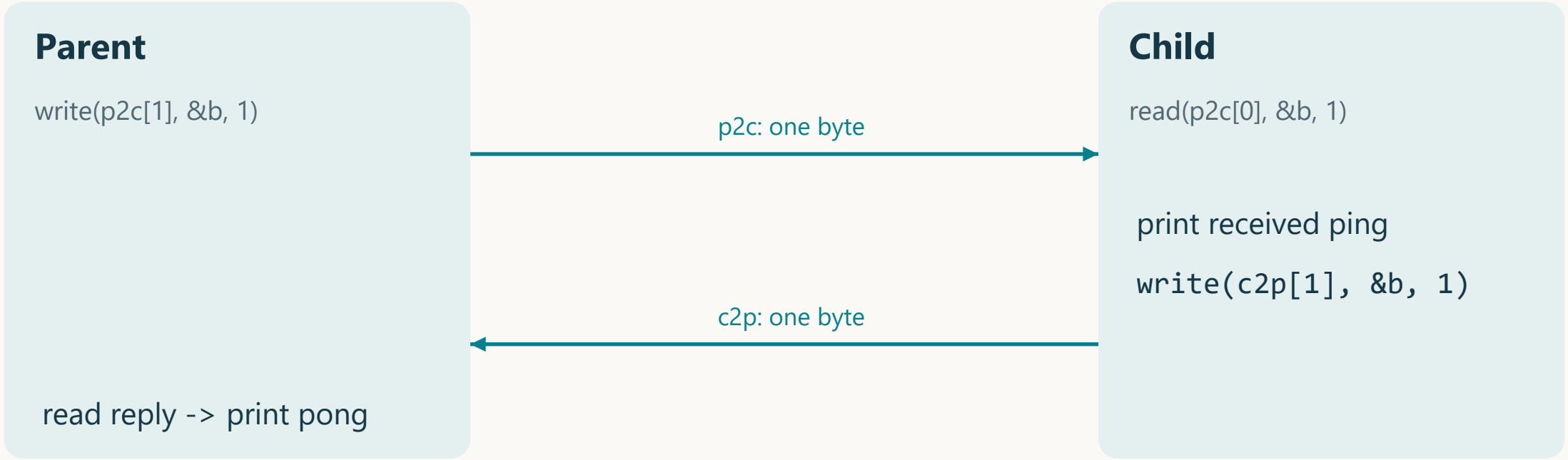
→

Kernel operation

proc.c / exec.c / pipe.c

The interface is small enough to trace from your C call to the kernel implementation.

pingpong: one pipe in each direction



The parent blocks in read(c2p[0], ...). The reply creates the ordering.

Each process owns only what it needs

Parent

```
Keeps: p2c[1], c2p[0]

Closes: p2c[0], c2p[1]
```

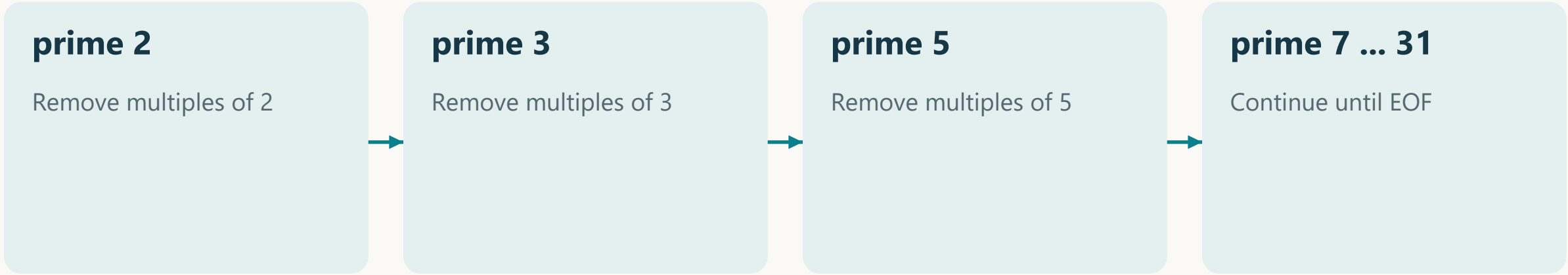
Child

```
Keeps: p2c[0], c2p[1]

Closes: p2c[1], c2p[0]
```

p2c = parent_to_child; c2p = child_to_parent in user/pingpong.c. Close the kept ends when their work is finished too.

An optional chain of prime filters



Each process takes its first number as p, then forwards only n % p != 0.

Input 2..35 -> output 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31

Lab 8: the observed behavior

run: parent 3, child 4

hello from the child | status 0

pingpong: child 10, parent 9

10: received ping -> 9: received pong

Variants and optional primes

ls: 0 | missing: 1 | primes: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31

Observed 19 October 2026 | RISC-V GCC 13.2.0 | QEMU 8.2.2

Earlier labs: names become file data



A hard link is another directory entry for the same inode. An open descriptor keeps an open-file reference even after unlink.

Demonstrate the required behavior

Linux

run -> zombie observation -> redirect -> pipeline

xv6

run -> pingpong -> optional primes

Evidence

Measured variants and outputs are in each lab report

Explain a changed condition

1. exec cannot find the requested program. What happens next?

2. The parent keeps a pipe write end. When can wc finish?

3. The child closes its reply pipe without writing. What does read return?